

Remove Duplicates from Sorted Array

Two approaches, visualized execution, and the core logic to remember

Brute Force · $O(n)$ space

Two Pointers · $O(1)$ space

Problem Statement

Given a sorted array `nums`, remove duplicates **in-place** so each unique value appears once, keeping relative order. Return `k`, the count of unique values — the first `k` positions of `nums` must hold the sorted unique values.



Approach Comparison

Approach	Time	Space	Core Idea
Brute Force	$O(n)$	$O(n)$	Build a clean temp array, copy back
Two Pointers	$O(n)$	$O(1)$	Slow pointer marks last unique slot, fast pointer scans

Big picture: Both approaches are $O(n)$ time, but the brute force pays for a second array. The two-pointer version does the same "only keep it if it's new" logic directly on top of the original array, satisfying the in-place requirement.

1 · Brute Force (Extra Space)

O(n) time

O(n) space

Seed temp[0]
with nums[0]



Scan nums;
new value? append



Copy temp
back over nums

```
// only write to temp when the value changes
private static int bruteforceSolution(int[] array) {
    int tempCounter = 0;
    int[] tempArray = new int[array.length];
    tempArray[tempCounter] = array[0];

    for (int i = 1; i < array.length; i++) {
        if (array[i] != tempArray[tempCounter]) {
            tempCounter++;
            tempArray[tempCounter] = array[i];
        }
    }

    for (int i = 0; i <= tempCounter; i++) {
        array[i] = tempArray[i];
    }

    return tempCounter + 1;
}
```

Recall trick: "Only write to temp when the value changes." Correct and easy to follow, but it uses O(n) extra space — the problem explicitly asks for an in-place solution.

2 · Two Pointers (In-Place)

O(n) time

O(1) space

previous = 0
current = 1



same value?
current++ only



new value?
write at previous+1



advance both,
count++

```
// slow pointer = last unique slot, fast pointer = scanner
private static int usingTwoPointers(int[] array) {
    int result = 1;
    int previous = 0;
    int current = 1;

    while (current < array.length) {
        if (array[current] == array[previous]) {
            current++;
            continue;
        }
        array[previous + 1] = array[current];
        current++;
        previous++;
        result++;
    }
    return result;
}
```

Trace for array = [0,0,1,1,1,2,2,3,3,4]:

current	array[current]	array[previous]	action	previous after	array state
1	0	0	equal → skip	0	[0,0,1,1,1,2,2,3,3,4]
2	1	0	write → array[1]=1	1	[0,1,1,1,1,2,2,3,3,4]
3	1	1	equal → skip	1	same
4	1	1	equal → skip	1	same
5	2	1	write → array[2]=2	2	[0,1,2,1,1,2,2,3,3,4]
6	2	2	equal → skip	2	same
7	3	2	write → array[3]=3	3	[0,1,2,3,1,2,2,3,3,4]
8	3	3	equal → skip	3	same
9	4	3	write → array[4]=4	4	done → k=5, [0,1,2,3,4,...]

Recall trick: "previous = last unique slot, current = scanner." When values match, just move the scanner. When they differ, place the new value right after the last unique one, then advance both. No extra array needed — duplicates get silently overwritten in place.

TL;DR — For Future Me

What to remember

- **Best solution:** Two pointers, $O(n)$ time, $O(1)$ space — "slow pointer marks the last unique slot, fast pointer scans and writes new values in."
- **Brute force** is correct but violates the in-place constraint by using a second array.
- **Underlying pattern:** this is the classic slow/fast pointer technique for in-place array compaction — it reappears in problems like removing a specific value, moving zeroes, or deduplicating with at-most-two-copies allowed.

Reference notes : [RemoveDuplicatesFromSortedArray.java](#)